test_pytest.py::test_01_get_customers
Test Case 1: Retrieve Customer List
Page being tested: http://127.0.0.1:5000/api/v1/customers
Inputs:
- Request Method: GET
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 200
- JSON: [{"customer_id": 1, "name": "Acme Corp", "company_name": "Acme Tractors"}]

Actual Output:
- HTTP Status Code: 200
- JSON: [{"customer_id": 1, "name": "Acme Corp", "company_name": "Acme Tractors"}]

Result: Success

PASSED

```
def test_01_get_cus Follow link (alt + click) database):
    url = "http://127.0.0.1:5000/api/v1/customers"
    response = client.get('/api/v1/customers')
    actual = response.get_json()
    expected = [{"customer_id": 1, "name": "Acme Corp", "company_name": "Acme Tractors"}]
    print_test_case(1, "Retrieve Customer List", url, "GET", None, 200, expected, response.status_code, actual)
    assert response.status_code == 200
```

test_pytest.py::test_02_create_enquiry
Test Case 2: Create a New Enquiry
Page being tested: http://127.0.0.1:5000/api/v1/enquiries
Inputs:
- Request Method: POST
- JSON: {"customer_id": 1}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 201
- JSON: {"message": "Enquiry created successfully", "enquiry_id": 2, "status": "New"}

Actual Output:
- HTTP Status Code: 201
- JSON: {"message": "Enquiry created successfully", "enquiry_id": 2, "status": "New"}

Result: Success

PASSED

```
def test_02_create_enquiry(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/enquiries"
    payload = {"customer_id": 1}
    response = client.post('/api/v1/enquiries', json=payload)
    actual = response.get_json()
    expected = {"message": "Enquiry created successfully", "enquiry_id": 2, "status": "New"}
    print_test_case(2, "Create a New Enquiry", url, "POST", json.dumps(payload), 201, expected, response.status_code, actual)
    assert response.status_code == 201
```

test_pytest.py::test_03_generate_quotation
Test Case 3: Generate Quotation
Page being tested: http://127.0.0.1:5000/api/v1/enquiries/2/quotations
Inputs:
- Request Method: POST
- JSON: {"price": 1200.5, "est_delivery": "2026-05-15"}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 201
- JSON: {"message": "Quotation generated", "quote_id": 2}

Actual Output:
- HTTP Status Code: 404
- JSON: {"error": {"code": 404, "message": "Enquiry ID 2 not found."}}

Result: Failed

FAILED

```
def test_03_generate_quotation(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/enquiries/2/quotations"
    payload = {"price": 1200.50, "est_delivery": "2026-05-15"}
    response = client.post('/api/v1/enquiries/2/quotations', json=payload)
    actual = response.get_json()
    expected = {"message": "Quotation generated", "quote_id": 2}
    print_test_case(3, "Generate Quotation", url, "POST", json.dumps(payload), 201, expected, response.status_code, actual)
    assert response.status_code == 201
```

test_pytest.py::test_04_revise_quotation
Test Case 4: Revise Quotation
Page being tested: http://127.0.0.1:5000/api/v1/quotations/1/revise
Inputs:
- Request Method: POST
- JSON: {"price": 1400.0}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 201
- JSON: {"message": "Quotation revised to version 2", "new_quote_id": 3}

Actual Output:
- HTTP Status Code: 201
- JSON: {"message": "Quotation revised to version 2", "new_quote_id": 2}

Result: Success

PASSED

```
def test_04_revise_quotation(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/quotations/1/revise"
    payload = {"price": 1400.00}
    response = client.post('/api/v1/quotations/1/revise', json=payload)
    actual = response.get_json()
    expected = {"message": "Quotation revised to version 2", "new_quote_id": 3}
    print_test_case(4, "Revise Quotation", url, "POST", json.dumps(payload), 201, expected, response.status_code, actual)
    assert response.status_code == 201
```

test_pytest.py::test_05_accept_quotation
Test Case 5: Accept Quotation (Create Order)
Page being tested: http://127.0.0.1:5000/api/v1/quotations/1/accept
Inputs:
- Request Method: POST
- JSON: {"spring_id": 1}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 201
- JSON: {"message": "Quotation accepted and Order confirmed", "order_id": 3}

Actual Output:
- HTTP Status Code: 201
- JSON: {"message": "Quotation accepted and Order confirmed", "order_id": 3}

Result: Success

PASSED

```python
def test_05_accept_quotation(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/quotations/1/accept"
    payload = {"spring_id": 1}
    response = client.post('/api/v1/quotations/1/accept', json=payload)
    actual = response.get_json()
    expected = {"message": "Quotation accepted and Order confirmed", "order_id": 3}
    print_test_case(5, "Accept Quotation (Create Order)", url, "POST", json.dumps(payload), 201, expected, response.status_code, actual)
    assert response.status_code == 201
```

test_pytest.py::test_06_get_materials
Test Case 6: Check Material Inventory
Page being tested: http://127.0.0.1:5000/api/v1/materials
Inputs:
- Request Method: GET
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 200
- JSON: [{"material_id": 1, "name": "Steel Wire", "specification": null, "stock_quantity": 100.0}]

Actual Output:
- HTTP Status Code: 200
- JSON: [{"material_id": 1, "name": "Steel Wire", "specification": null, "stock_quantity": 100.0}]

Result: Success

PASSED

```python
def test_06_get_materials(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/materials"
    response = client.get('/api/v1/materials')
    actual = response.get_json()
    expected = [{"material_id": 1, "name": "Steel Wire", "specification": None, "stock_quantity": 100.0}]
    print_test_case(6, "Check Material Inventory", url, "GET", None, 200, expected, response.status_code, actual)
    assert response.status_code == 200
```

test_pytest.py::test_07_schedule_task
Test Case 7: Schedule Task (Epic 2)
Page being tested: http://127.0.0.1:5000/api/v1/tasks
Inputs:
- Request Method: POST
- JSON: {"order_id": 3, "machine_id": 1, "material_id": 1, "scheduled_at": "2026-05-01T10:00:00", "material_required": 20.0}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 201
- JSON: {"message": "Production task scheduled and materials reserved", "task_id": 1}

Actual Output:
- HTTP Status Code: 201
- JSON: {"message": "Production task scheduled and materials reserved", "task_id": 1}

Result: Success

PASSED

```python
def test_07_schedule_task(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/tasks"
    payload = {"order_id": 3, "machine_id": 1, "material_id": 1, "scheduled_at": "2026-05-01T10:00:00", "material_required": 20.0}
    response = client.post('/api/v1/tasks', json=payload)
    actual = response.get_json()
    expected = {"message": "Production task scheduled and materials reserved", "task_id": 1}
    print_test_case(7, "Schedule Task (Epic 2)", url, "POST", json.dumps(payload), 201, expected, response.status_code, actual)
    assert response.status_code == 201
```

test_pytest.py::test_08_update_task_status
Test Case 8: Update Production Progress
Page being tested: http://127.0.0.1:5000/api/v1/tasks/1/status
Inputs:
- Request Method: PUT
- JSON: {"status": "In Progress"}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 200
- JSON: {"message": "Task status updated to In Progress"}

Actual Output:
- HTTP Status Code: 200
- JSON: {"message": "Task status updated to In Progress"}

Result: Success

PASSED

```python
def test_08_update_task_status(client, init_database):
    # Setup: Call schedule task to ensure task 1 exists
    client.post('/api/v1/tasks', json={"order_id": 3, "machine_id": 1, "material_id": 1, "scheduled_at": "2026-05-01T10:00:00", "material_required": 10.0})

    url = "http://127.0.0.1:5000/api/v1/tasks/1/status"
    payload = {"status": "In Progress"}
    response = client.put('/api/v1/tasks/1/status', json=payload)
    actual = response.get_json()
    expected = {"message": "Task status updated to In Progress"}
    print_test_case(8, "Update Production Progress", url, "PUT", json.dumps(payload), 200, expected, response.status_code, actual)
    assert response.status_code == 200
```

test_pytest.py::test_09_quality_control
Test Case 9: Submit Quality Report
Page being tested: http://127.0.0.1:5000/api/v1/tasks/1/quality
Inputs:
- Request Method: POST
- JSON: {"inspector": "Mike Foreman", "result": "Approved"}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 201
- JSON: {"message": "Quality report recorded: Approved", "report_id": 1}

Actual Output:
- HTTP Status Code: 201
- JSON: {"message": "Quality report recorded: Approved", "report_id": 1}

Result: Success

PASSED

```python
def test_09_quality_control(client, init_database):
    # Ensure task 1 exists
    client.post('/api/v1/tasks', json={"order_id": 3, "machine_id": 1, "material_id": 1, "scheduled_at": "2026-05-01T10:00:00", "material_required": 10.0})

    url = "http://127.0.0.1:5000/api/v1/tasks/1/quality"
    payload = {"inspector": "Mike Foreman", "result": "Approved"}
    response = client.post('/api/v1/tasks/1/quality', json=payload)
    actual = response.get_json()
    expected = {"message": "Quality report recorded: Approved", "report_id": 1}
    print_test_case(9, "Submit Quality Report", url, "POST", json.dumps(payload), 201, expected, response.status_code, actual)
    assert response.status_code == 201
```

test_pytest.py::test_10_generate_invoice
Test Case 10: Generate Invoice for Order
Page being tested: http://127.0.0.1:5000/api/v1/orders/1/invoice
Inputs:
- Request Method: POST
- JSON: {"manual_amount": 1050.0}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 201
- JSON: {"message": "Invoice generated successfully", "invoice_id": 2, "amount": 1050.0}

Actual Output:
- HTTP Status Code: 201
- JSON: {"message": "Invoice generated successfully", "invoice_id": 2, "amount": 1050.0}

Result: Success

PASSED

```python
def test_10_generate_invoice(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/orders/1/invoice"
    payload = {"manual_amount": 1050.0}
    response = client.post('/api/v1/orders/1/invoice', json=payload)
    actual = response.get_json()
    expected = {"message": "Invoice generated successfully", "invoice_id": 2, "amount": 1050.0}
    print_test_case(10, "Generate Invoice for Order", url, "POST", json.dumps(payload), 201, expected, response.status_code, actual)
    assert response.status_code == 201
```

test_pytest.py::test_11_pay_invoice
Test Case 11: Mark Invoice as Paid
Page being tested: http://127.0.0.1:5000/api/v1/invoices/1/pay
Inputs:
- Request Method: PUT
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 200
- JSON: {"message": "Invoice successfully marked as paid"}

Actual Output:
- HTTP Status Code: 200
- JSON: {"message": "Invoice successfully marked as paid"}

Result: Success

PASSED

```python
def test_11_pay_invoice(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/invoices/1/pay"
    response = client.put('/api/v1/invoices/1/pay')
    actual = response.get_json()
    expected = {"message": "Invoice successfully marked as paid"}
    print_test_case(11, "Mark Invoice as Paid", url, "PUT", None, 200, expected, response.status_code, actual)
    assert response.status_code == 200
```

test_pytest.py::test_12_dispatch_shipment
Test Case 12: Dispatch Shipment
Page being tested: http://127.0.0.1:5000/api/v1/orders/2/shipment
Inputs:
- Request Method: POST
- JSON: {"carrier": "UPS", "tracking_number": "UPS987654"}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 201
- JSON: {"message": "Order dispatched", "shipment_id": 2}

Actual Output:
- HTTP Status Code: 201
- JSON: {"message": "Order dispatched", "shipment_id": 2}

Result: Success

PASSED

```python
def test_12_dispatch_shipment(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/orders/2/shipment"
    payload = {"carrier": "UPS", "tracking_number": "UPS987654"}
    response = client.post('/api/v1/orders/2/shipment', json=payload)
    actual = response.get_json()
    expected = {"message": "Order dispatched", "shipment_id": 2} # ID 2 because fixture has shipment 1
    print_test_case(12, "Dispatch Shipment", url, "POST", json.dumps(payload), 201, expected, response.status_code, actual)
    assert response.status_code == 201
```

test_pytest.py::test_13_record_delivery
Test Case 13: Record Delivery Status
Page being tested: http://127.0.0.1:5000/api/v1/shipments/1/delivery
Inputs:
- Request Method: PUT
- JSON: {"delivery_status": "Delivered & Accepted"}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 200
- JSON: {"message": "Delivery updated: Delivered & Accepted"}

Actual Output:
- HTTP Status Code: 200
- JSON: {"message": "Delivery updated: Delivered & Accepted"}

Result: Success

PASSED

```python
def test_13_record_delivery(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/shipments/1/delivery"
    payload = {"delivery_status": "Delivered & Accepted"}
    response = client.put('/api/v1/shipments/1/delivery', json=payload)
    actual = response.get_json()
    expected = {"message": "Delivery updated: Delivered & Accepted"}
    print_test_case(13, "Record Delivery Status", url, "PUT", json.dumps(payload), 200, expected, response.status_code, actual)
    assert response.status_code == 200
```

test_pytest.py::test_14_financial_analytics
Test Case 14: Get Financial Analytics
Page being tested: http://127.0.0.1:5000/api/v1/analytics/financials
Inputs:
- Request Method: GET
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 200
- JSON: {"metrics": {"total_revenue": 1500.0, "collected_revenue": 0.0, "pending_revenue": 1500.0}, "chart_data": {"labels": ["Collected Revenue", "Pending Revenue"], "series": [0.0, 1500.0]}}

Actual Output:
- HTTP Status Code: 200
- JSON: {"metrics": {"total_revenue": 1500.0, "collected_revenue": 0.0, "pending_revenue": 1500.0}, "chart_data": {"labels": ["Collected Revenue", "Pending Revenue"], "series": [0.0, 1500.0]}}

Result: Success

PASSED

```python
def test_14_financial_analytics(client, init_database):
    # Note: Fixture creates 1 unpaid invoice for $1500. We just paid it in test_11 logic, but fixtures reset per test!
    url = "http://127.0.0.1:5000/api/v1/analytics/financials"
    response = client.get('/api/v1/analytics/financials')
    actual = response.get_json()
    expected = {
        "metrics": {"total_revenue": 1500.0, "collected_revenue": 0.0, "pending_revenue": 1500.0},
        "chart_data": {"labels": ["Collected Revenue", "Pending Revenue"], "series": [0.0, 1500.0]}
    }
    print_test_case(14, "Get Financial Analytics", url, "GET", None, 200, expected, response.status_code, actual)
    assert response.status_code == 200
```

test_pytest.py::test_15_production_analytics
Test Case 15: Get Production Analytics
Page being tested: http://127.0.0.1:5000/api/v1/analytics/production
Inputs:
- Request Method: GET
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 200
- JSON: {"machine_utilization": {"chart_type": "bar", "labels": [], "data": []}, "production_pipeline": {"chart_type": "pie", "data": {"Invoiced - Pending Dispatch": 1, "Quality Approved - Ready for Billing": 1}}}

Actual Output:
- HTTP Status Code: 200
- JSON: {"machine_utilization": {"chart_type": "bar", "labels": [], "data": []}, "production_pipeline": {"chart_type": "pie", "data": {"Invoiced - Pending Dispatch": 1, "Quality Approved - Ready for Billing": 1}}}

Result: Success

PASSED

```python
def test_15_production_analytics(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/analytics/production"
    response = client.get('/api/v1/analytics/production')
    actual = response.get_json()

    # We expect our 2 fixture orders in the pipeline breakdown
    expected = {
        "machine_utilization": {"chart_type": "bar", "labels": [], "data": []},
        "production_pipeline": {"chart_type": "pie", "data": {"Invoiced - Pending Dispatch": 1, "Quality Approved - Ready for Billing": 1}}
    }
    print_test_case(15, "Get Production Analytics", url, "GET", None, 200, expected, response.status_code, actual)
    assert response.status_code == 200
```

test_pytest.py::test_16_validation_failure
Test Case 16: Validation Check (Not Found)
Page being tested: http://127.0.0.1:5000/api/v1/orders/3/invoice
Inputs:
- Request Method: POST
- JSON: {}
- Header: Content-Type: application/json

Expected Output:
- HTTP Status Code: 404

- JSON: {"error": {"code": 404, "message": "Order ID 3 not found"}}

Actual Output:

- HTTP Status Code: 404
- JSON: {"error": {"code": 404, "message": "Order ID 3 not found"}}

Result: Success

PASSED

```python
def test_16_validation_failure(client, init_database):
    url = "http://127.0.0.1:5000/api/v1/orders/3/invoice"
    # Order 3 does not exist in fixture, should return 404
    response = client.post('/api/v1/orders/3/invoice', json={})
    actual = response.get_json()
    expected = {"error": {"code": 404, "message": "Order ID 3 not found"}}
    print_test_case(16, "Validation Check (Not Found)", url, "POST", "{}", 404, expected, response.status_code, actual)
    assert response.status_code == 404
```